

CSCE 606 – Software Engineering
Project: Inventory System
Final Report

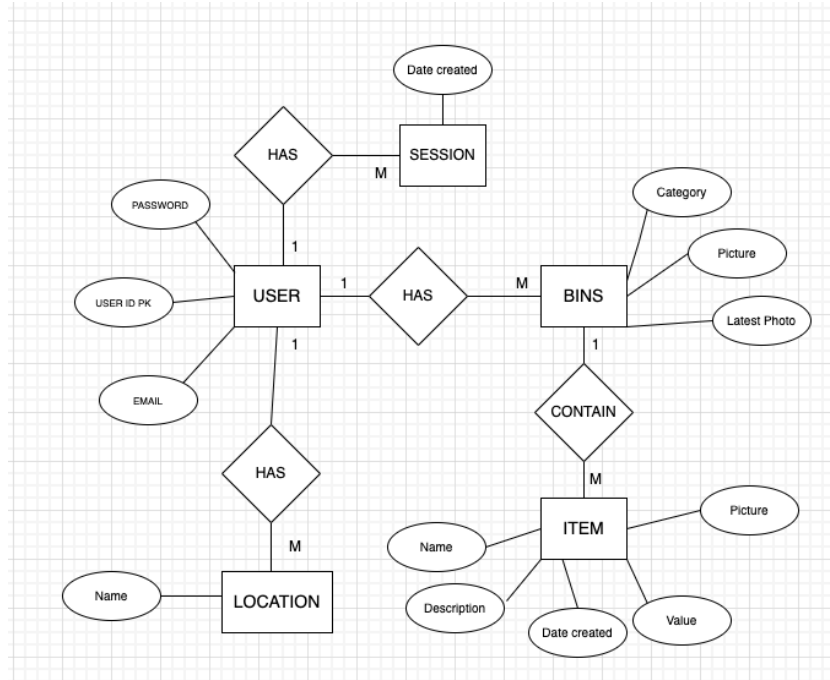
1 - Summary of the project

Our customer, Dr. Robert Lightfoot, expressed a clear need for a server-based application to help him efficiently organize and manage the contents of his garage and multiple storage locations. His initial idea was to make use of a surplus of NFC chips by attaching them to bins and containers. He wanted a system where, by scanning an NFC chip, he could instantly access information about the contents of that specific bin. In addition to bin-level organization, he needed to track uncontained items like large tools or a bike and include them in the digital inventory. Furthermore, Dr. Lightfoot required search functionality that allowed him to find items by name, category, or date, along with the ability to log newly added items in a printable format. He also requested features to help track items intended for garage sales and to enable sharing of specific bins with collaborators, such as his wife, for joint management.

To meet these requirements, we developed a user-friendly, informative web application using Ruby on Rails, following Agile methodology to iteratively refine and deliver features based on feedback. Our application integrates NFC scanning functionality, supports inventory management across multiple storage locations, and includes robust search, logging, and sharing features. Compared to Iteration 0, where the idea was still loosely defined, this final implementation delivers a concrete, scalable solution tailored to the customer's daily organizational challenges.

2- User Stories

For Sprint #1, we focused on laying the foundation of our app, particularly the creation of models and the database. Since the first customer meeting, we gathered several user stories provided by the client and began building the models based on those. In the following sprints, we encountered several challenges with our original design—especially after introducing a Location model that was tied to the user rather than to bins and items. As a result, several of our initial models had to be updated along the way. Another important aspect of our app was the UI design, which needed to be responsive and user-friendly, especially on mobile devices. Below is the final ER diagram of our app:



All the user stories mentioned below were implemented and completed.

2.1 USER Model:

An important aspect of our app is authentication. Initially, we used the built-in authentication features provided by Rails. However, after Sprint 2, we realized this approach was not sufficient. We wanted to add additional features such as password reset capabilities and more robust account management.

To address this, we opted to use Devise, a widely used gem for authentication in Rails applications.

- **Feature: Create my account for the first time**

As a new user of the website.

I want to create my account

So that I can start adding items to my inventory

Points: 3

This story was originally assigned 3 points; however, after we switched to Devise, we refactored the user story and increased its score to 5—mainly due to all the additional changes and configurations required.

- **Feature: Log in to my account**

As a user with many inventories
I want to log in to my account to have access to my homepage
So that I can manage all the bins and stuff inside them
Points: 3

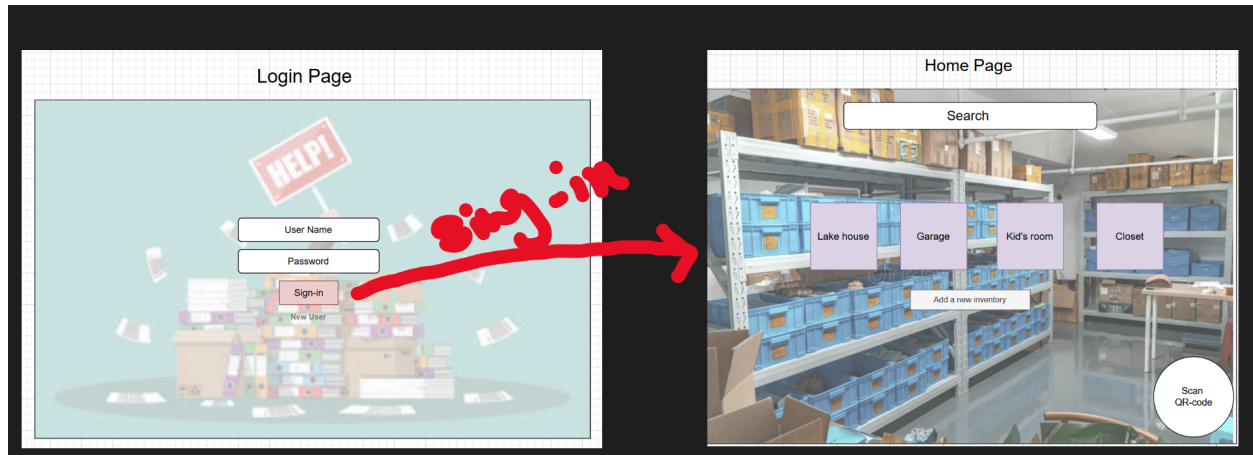
- **Feature: Password Recovery**

As a user,
I want to reset my password,
So that I can recover my account if I forget my credentials.
Points: 4

As part of Sprint 2, as previously mentioned, we decided to strengthen our authentication system and switched to Devise. This process turned out to be more challenging than we initially expected. We spent a considerable amount of time improving these features. One of the most complex parts was implementing password reset functionality. We decided to handle it via email, which introduced another layer of complexity. Specifically, we had to set up an SMTP email account and integrate it with our application. Without a doubt, this user story became one of the most difficult to implement. During the sprint retrospective, we revised its score to 5.

USER UI Mock-UP

This was the original mock up:



At the beginning, we had only a vague idea of how the login page would look. Later on, we refined and improved the UI, as shown in the screenshot below:

 **INVENTORY SYSTEM**

Welcome Back

 Email

 Password

 [Forgot Password?](#)

SIGN IN

Don't have an account?  [Sign Up](#)

Forgot Password

Enter your registered email:

Send Reset Code

[Back to Login](#)

Enter Reset Code

Reset Code:

Verify

Resend in 26 seconds

Reset Password

New Password 

Confirm Password:

Reset Password

Sign Up

Name

Email

Password ?

Password confirmation

Sign Up

Already have an account? [Login here](#)

2.2 BIN Model:

This feature is the core of our app and the main priority for our customer. He wanted an application to manage inventory composed of bins distributed across different locations. Two key requests made this app unique: the ability to scan a QR code or use NFC to quickly look up the contents of a bin.

Similar to the user model, we started with a basic implementation for bins. However, we later realized the model needed improvement—particularly, the location attribute. Initially, location was part of the bin model, but we decided to refactor it into a separate location model associated with the user. This change gave users better control and organization of bins across multiple locations.

An important addition during Sprint 3 was the ability to share bins with family members. This feature was successfully implemented and included in the final product.

- **Feature: Add/update/delete a bin**

As a user,

I want to view and update existing bins or add new ones to the database,

So that I can efficiently manage and expand my storage for items as needed.

Points: 3

This was the first user story implemented for bins and included basic CRUD operations. We had to refactor this story multiple times, especially after encountering scenarios that required special handling, for example: Can a user delete a bin that still contains items?. According to our ER diagram, bins belong to individual users and are always associated with a location. Several validations were added after Sprint 1 to enforce these constraints, which made the model more robust and reduced the likelihood of bugs.

The original point estimate for this story was 3, but due to the necessary refactors and edge case handling, its complexity increased significantly.

- **Feature: List of pictures of items in a Bin**

As a logged-in user

I want to click on “What is inside the bin”

So that I can have a picture of all the items in the bin

Points: 5

- **Feature: Create Picture Model**

As a logged-in user,

I want to take and attach pictures to bins and locations,

So that I can easily check what is inside a bin.

Points: 5

These two user stories are interrelated. The inclusion of pictures in our model was necessary from the beginning; however, it introduced additional complexity, particularly regarding how images would be stored. To avoid this complexity and the need for an external cloud storage service, we decided to remove the separate Picture model and instead allow only one picture per bin and one per item.

The built-in Active Storage functionality in Rails proved to be extremely useful and simplified the implementation of these user stories.

- **Feature: Add QR Code to Bin**

As a logged-in user,

I want to have the option to add a QR code when creating a bin,

So that I can scan it and easily check its contents.

Points: 3

- **Feature: Scan QR Code for Bin Information**

As a logged-in user,

I want to scan a QR code,

So that I can quickly check the bin's contents.

Points: 3

- **Feature:** Add NFC Chip to Bin
As a logged-in user,
I want to attach an NFC chip to a bin,
So that I can scan it to find information about its contents.

Points: 5

How does a user quickly look up what's inside a bin? The answer is by scanning the bin using a QR code or an NFC chip.

QR Code Implementation:

Implementing QR codes was straightforward thanks to the `qr` gem, a simple QR code generator for Rails. The customer clearly outlined the desired flow: every time a bin is created, the system generates a unique QR code that links directly to the bin's URL. The user can then print this QR code and attach it to the bin.

When a user wants to check the contents of a bin, they simply scan the QR code using their phone, which opens the bin's page in the browser. The most complex part of this feature was securing access. We had to ensure that the bin being scanned actually belongs to the authenticated user. This added a layer of validation logic and increased the implementation complexity. We spent a considerable amount of time addressing edge cases and securing this feature properly.

NFC Tag Flow:

For NFC (Near Field Communication), the flow differs slightly. After a bin is created, users have the option to write the bin's URL to an NFC chip. To simplify this process, we added a "Copy URL" button on the bin show page.

Users can then use an external tool like NFC Tools on iOS or Android to paste and write the copied bin URL onto an NFC tag. Once programmed, scanning the NFC tag with a phone will open the same bin page as the QR code. This dual approach gives users flexibility—QR codes for printed labeling, and NFC for faster, touch-based access.

- **Feature:** Improve Bin Form with Dropdowns
As a logged in user,
I want to select locations and categories from the dropdowns in the bin form,
So that I can avoid creating duplicate entries.

Points: 3

- **Feature:** Search Bar on Dashboard
As a user of the website,
I want to have a search box,
So that I can quickly find my bins and items
Points: 5

An important feature of our app is the ability to search through bins efficiently. Once the user is on their dashboard, they can simply start typing in the search box, and the website dynamically looks up matching bins in real-time.

This search functionality was implemented using JavaScript, allowing for a smooth and responsive user experience. The search algorithm supports matching by both bin name and category, giving users flexibility when locating specific bins. This feature significantly enhances usability, especially for users with large inventories spread across multiple categories.

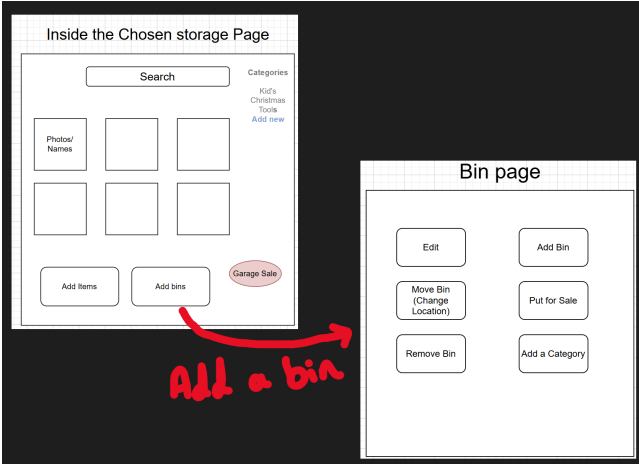
- **Feature:** UI for Bins
As a user of the website,
I want an organized table displaying all bins,
So that I can easily identify and manage my storage bins.
Points: 3

The UI for bins was redesigned several times based on customer feedback after each sprint. Initially, the focus was on basic functionality and layout. However, as we progressed, the customer emphasized the need for a clean, responsive design, especially for mobile devices.

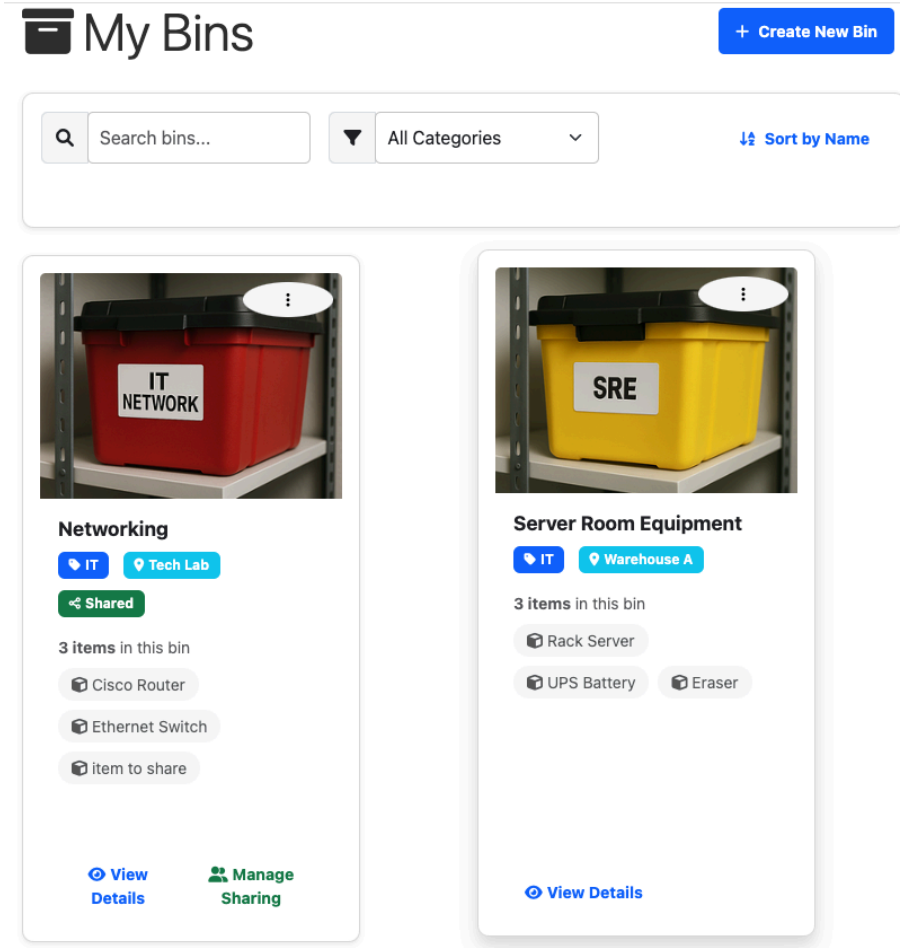
We re-evaluated the original point estimate from a lower value to 5 points due to the additional time spent on optimizing layout, implementing mobile responsiveness, and validating usability across different screen sizes.

BIN UI Mock-UP

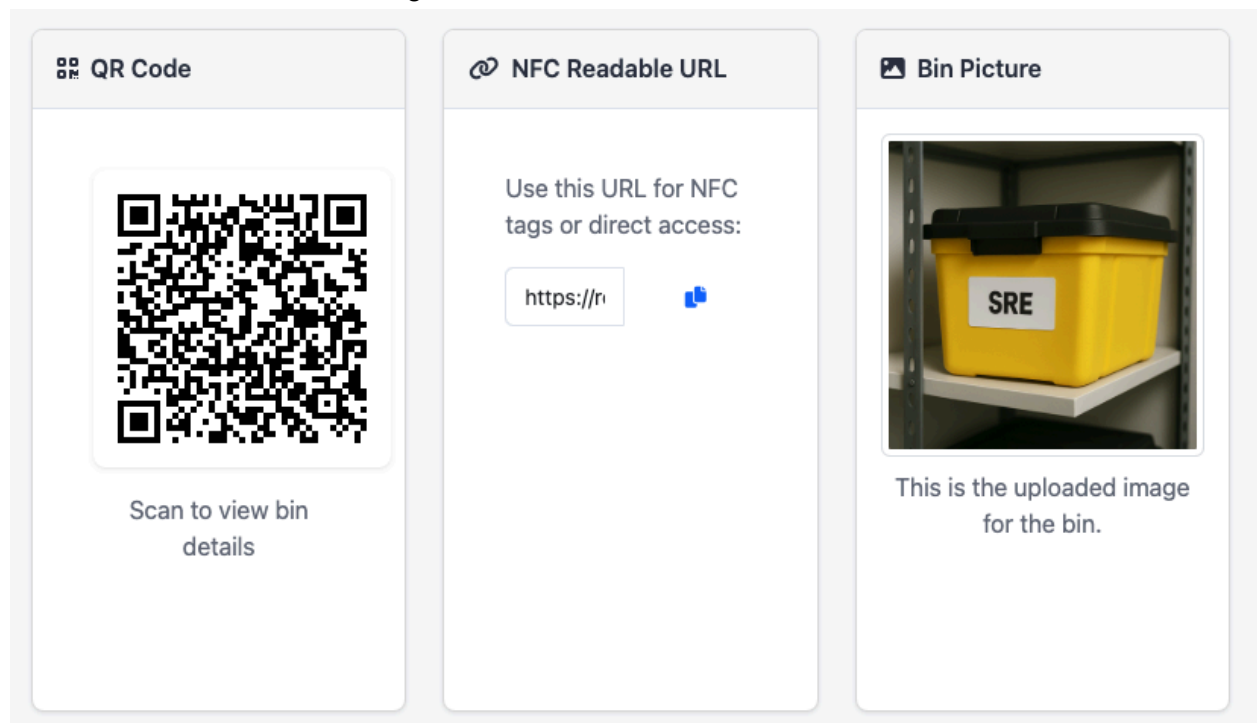
Original Mockup Diagram for bins:



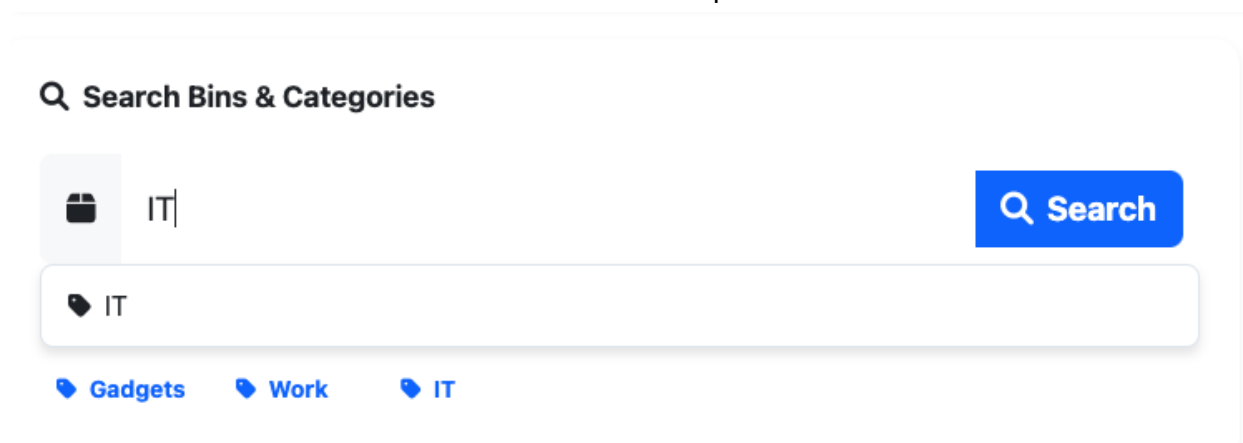
Current UI



Screenshot from /bin/x showing the QR code and NFC link:



Screenshot from /dashboard that shows the Search capabilities:



2.3 ITEM Model:

Users can store items inside bins, but this part of the app required significant refactoring. Initially, during Sprint 1, the Item model was created to strictly belong to a Bin. However, we quickly realized this was limiting. Large items such as bicycles or furniture cannot physically fit in a bin. To solve this, we refactored the model so that items belong directly to a user, allowing

users to create and manage standalone items that don't need to be inside a bin. With that being said, the relationship between bins and items still exists; a bin can contain multiple items, maintaining the structure needed for most inventory management tasks.

- **Feature: Add/update/delete an Item**

As a user,

I want to view and update existing items or add new ones to the database,

So that I can efficiently manage and expand my storage for items as needed.

Points: 5

This user story was originally completed during Sprint 1, but as previously mentioned, it had to be refactored several times. The challenge wasn't just about updating the model or creating new migrations; in fact, those parts were relatively straightforward. The difficulty came from having to adjust both our TDD (Test-Driven Development) and BDD (Behavior-Driven Development) test stories. Throughout the course, we found ourselves going back and forth between refactoring the models and updating the corresponding tests to reflect the new database schema and logic. This added extra complexity and time to what initially seemed like a simple task.

- **Feature: Prevent Item Deletion after removal from Bin**

As a logged-in user,

I want to retain item records after removing them from a bin,

So that I can access this information later.

Points: 4

- **Feature: Allow items that don't belong to any Bin**

As a logged-in user,

I want to add items that are not assigned to any bin,

So that I can keep track of larger items like bicycles.

Points: 2

These two stories summarize several user stories related to the Item model. We had multiple discussions with the customer to clarify how item deletions should be handled. Based on their feedback, we decided to implement the following behavior: If the item is associated with a bin, it can be deleted from the /bins or /items view, and will simply be removed from that bin. If the item is unassigned (i.e., it doesn't belong to any bin), it will be permanently deleted. To help users

understand the consequences of their actions, we implemented flash warning messages using Rails' built-in messaging system.

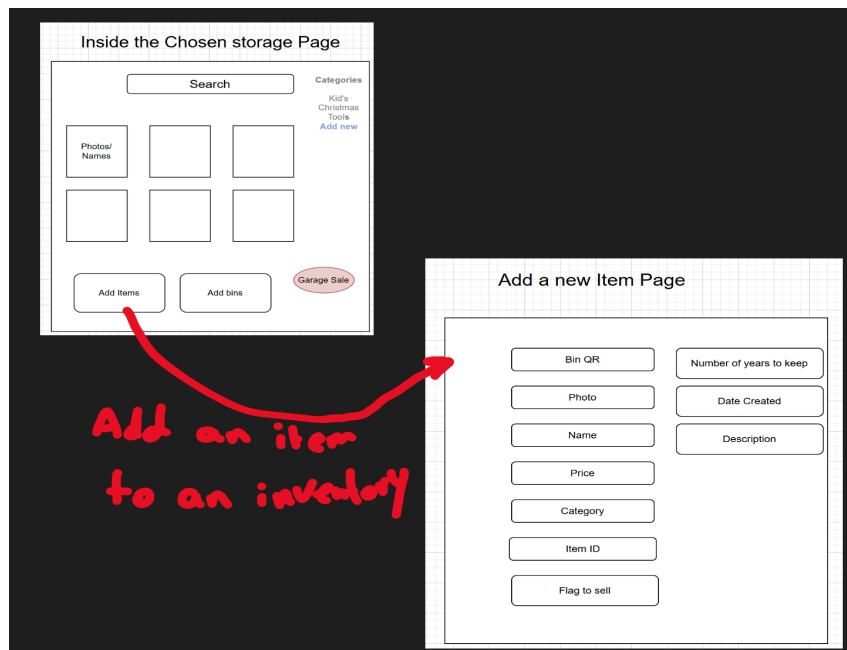
Originally, we assigned 2 points to this user story, underestimating its complexity. In reality, implementing the logic required significant changes to the Item model, which caused a cascade of issues in both our TDD and BDD test suites. This story turned out to be more challenging than anticipated.

- **Feature:** UI for Items
As a user of the website,
I want an organized table displaying all items,
So that I can easily identify and manage my inventory.
Points: 2

We made several changes to this user story throughout development. The final version features a mobile-friendly layout with intuitive icons representing each CRUD action.

ITEM UI Mock-UP

Original design:



Current Design:

HOME
BINS
ITEMS
FRIENDS
LOG
SALE

A

My Items

+ New Item

☐ For Sale Only

Reset

Owned

Rack Server

Server Room Equipment

Warehouse A

\$2,500.00

Owned

Cisco Router

Networking

Tech Lab

\$1,500.00

Owned

Ethernet Switch

Networking

Tech Lab

\$200.00

Shared

staple

Unassigned

Warehouse A

\$10.00

Owned

Eraser

Server Room Equipment

Warehouse A

\$10.00

Shared

Item with no Bin

Unassigned

Living Room

\$12.00

2.4 LOCATION Model:

A user can have multiple locations, such as Garage, Warehouse, or Main Closet. We implemented the Location model in Sprint 3 after realizing that we needed a dedicated model instead of relying on location attributes within /bins.

This addition, while necessary, introduced breaking changes in our existing tests and relationships. Since several parts of the application were already built under the assumption that bins handled their own location logic, integrating the new Location model required refactoring multiple models and updating test cases. We spent a significant amount of time resolving these issues to ensure a smooth transition and maintain application integrity.

- **Feature: Add/update/delete location**

As a user,

I want to view and update existing locations or add new ones to the database,

So that I can efficiently access all the storage I have

Points: 5

- **Feature: Search by Location on Dashboard**

As a user of the website,

I want to search for bins and items by location,

So that I can easily find items in specific locations.

Points: 3

- **Feature: Location Index Page**

As a user of the website,

I want to see a list of all available locations,

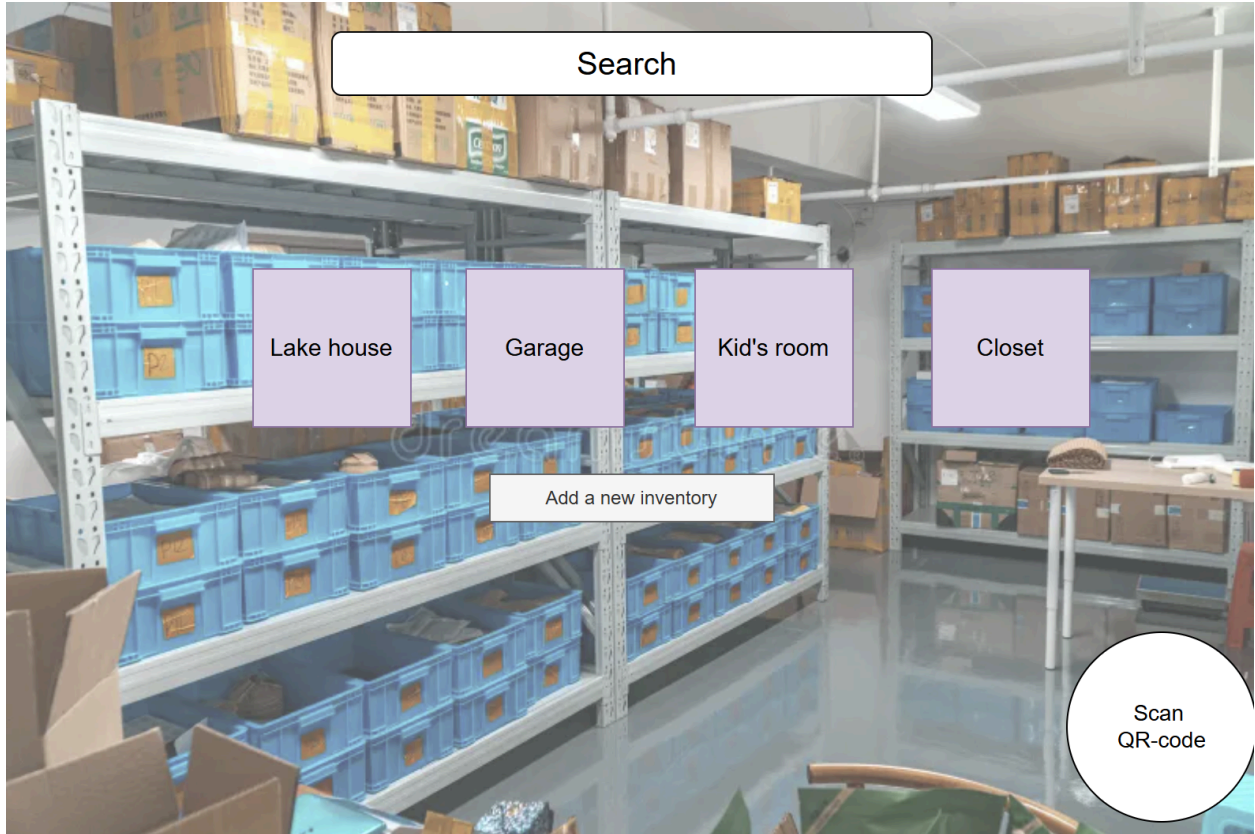
So that I can search for bins or items within a specific location.

Points: 1

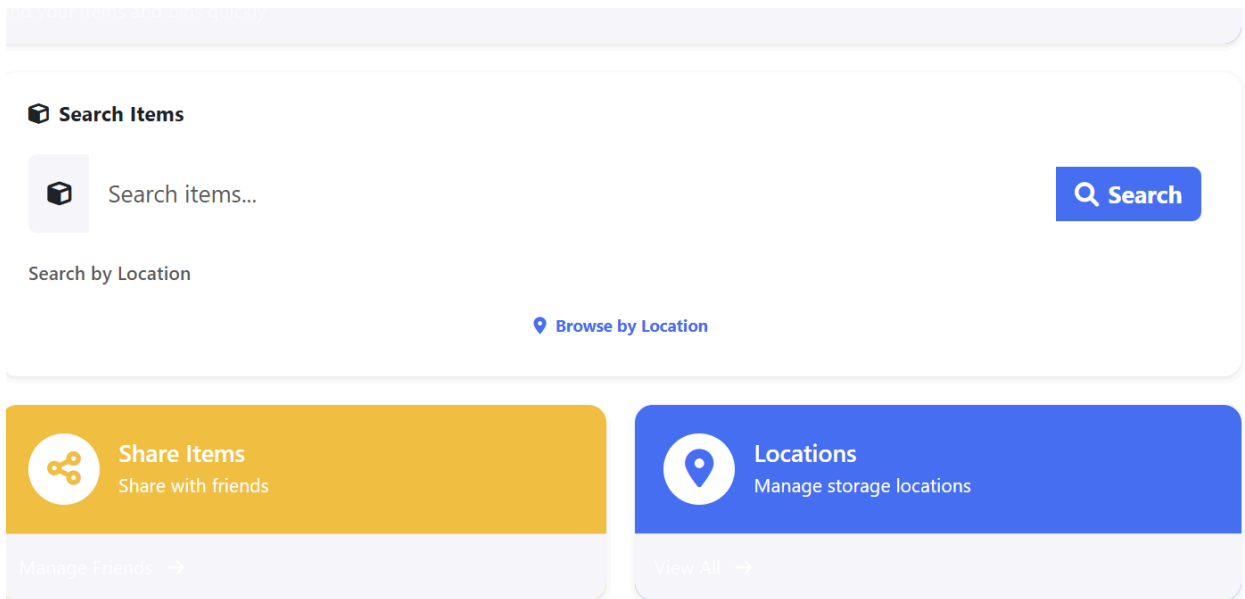
Our initial UI design featured a homepage displaying all storage locations. Users could click on any location to view the bins and individual items stored there, with options to add, delete, or edit content as needed. Additionally, the homepage included functionality to add new storage locations directly. The initial UI mockups reflect this layout and interaction flow. However, in our final design, the homepage offers users a broader set of capabilities and a more streamlined experience. Accessing storage locations is now one of several options available on the homepage. Once the user selects the “Locations” option, they can choose a specific location to view its associated bins and items. This structure improves navigation and makes the interface more scalable and user-friendly.

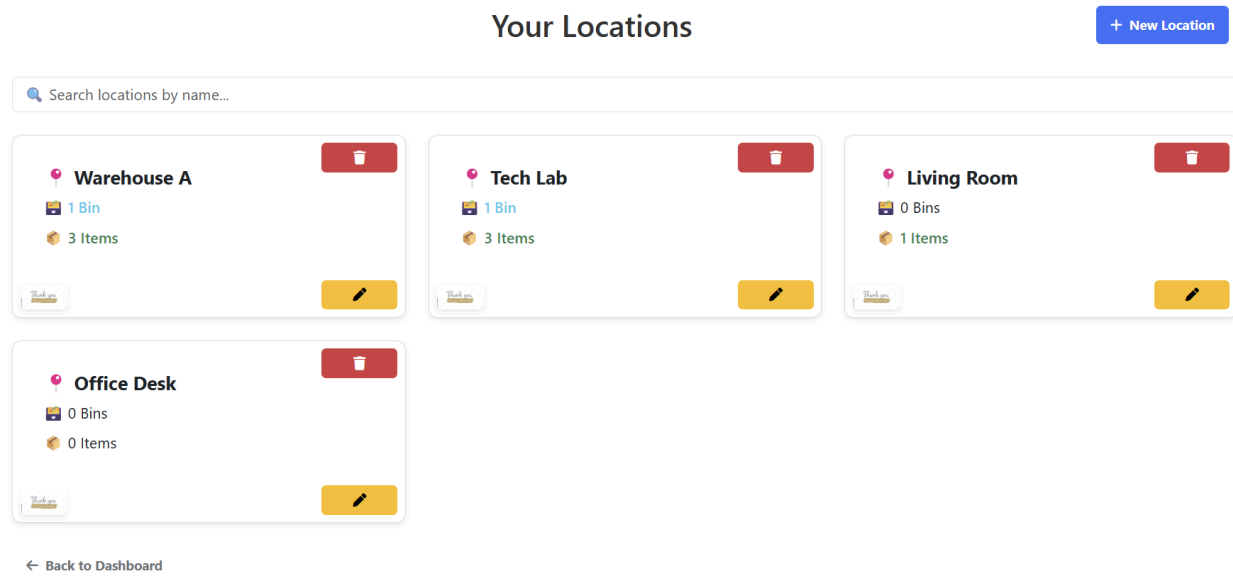
Location UI Mock-Up

Original design:



Current design:





2.5 Dashboard:

As mentioned in the previous section, our initial idea was to structure the homepage around displaying all storage locations. However, after using the web application ourselves and simulating real user behavior, we realized that users often don't know exactly where a specific item or bin is located. This insight led us to rethink our approach. To make the app more user-friendly, we prioritized the most frequently used functionalities, such as searching for bins and items regardless of location, and placed them directly on the homepage for quicker and easier access. The initial and final designs of our main dashboard reflect this shift in focus toward usability and efficiency.

However, this change in UI design, introduced during Sprints 3 and 4, created several challenges in maintaining and updating our Behavior-Driven Development (BDD) test scenarios. Many of our earlier BDDs were closely tied to the initial interface flow, so altering the structure of the homepage meant that numerous step definitions and feature files had to be rewritten or significantly revised. This experience taught us the importance of designing flexible, high-level BDDs that focus on user intentions rather than rigid UI flows, which would make the tests more resilient to design changes in future iterations.

- **Feature:** Search Bar on Dashboard
As a user of the website,
I want to have a search box,
So that I can quickly find my bins and items

Points: 5

Feature: Search by Location on Dashboard

As a user of the website,

I want to search for bins and items by location,

So that I can easily find items in specific locations.

Points: 3

This functionality has been successfully implemented. Users can now search by item name or bin name, with smart suggestions provided to assist in locating items even if the full name is not known. The final version also supports searching by location and category. To enable this, each bin is now assigned to a category. For bins that don't clearly fit into any specific group, a default category called "Others" is available to help users maintain a consistent organizational structure.

Feature: Mini Window for Logout

As a user of the website,

I want a mini window displaying my user information,

So that I can safely log out of the app.

Points: 2

Feature: Webpage Tab

As a user of the website,

I want to have a tab on top of the page,

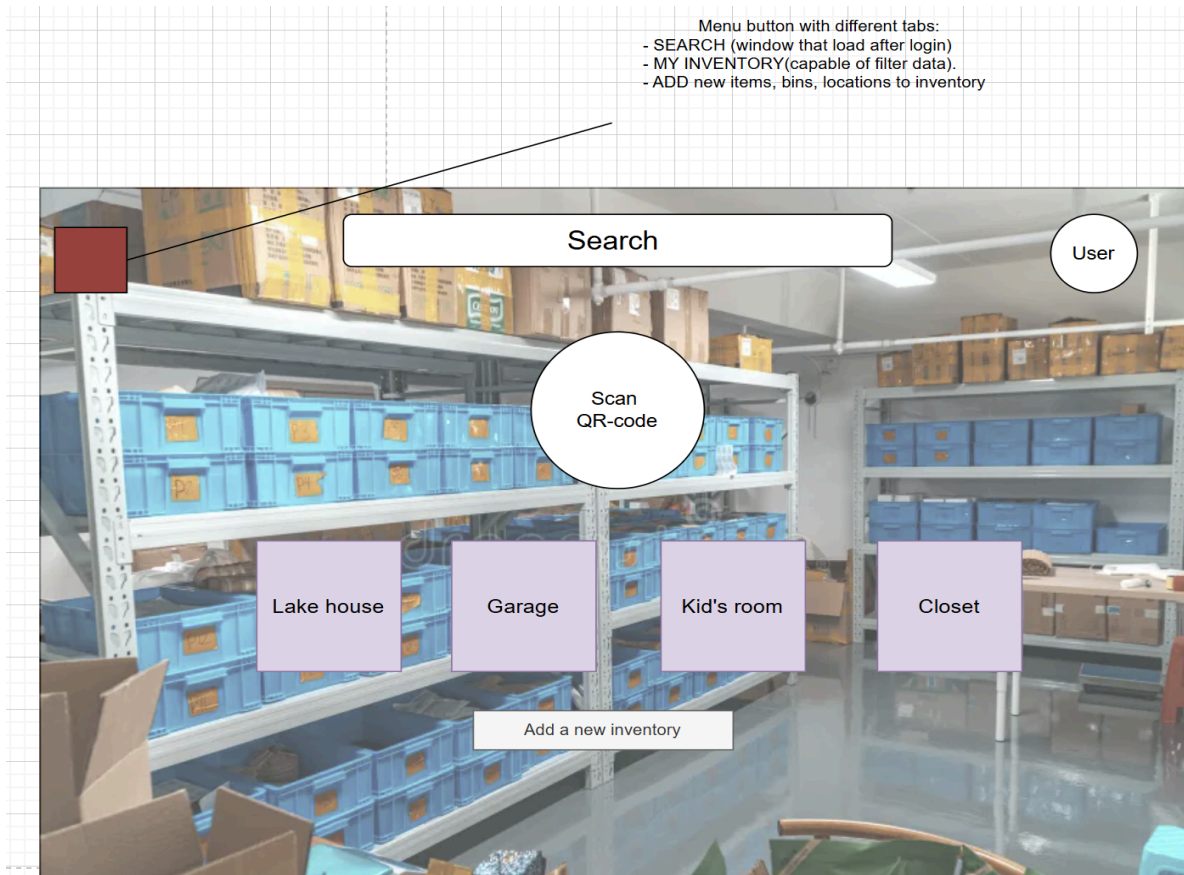
So that I can always perform any action with bins and items.

Points: 1

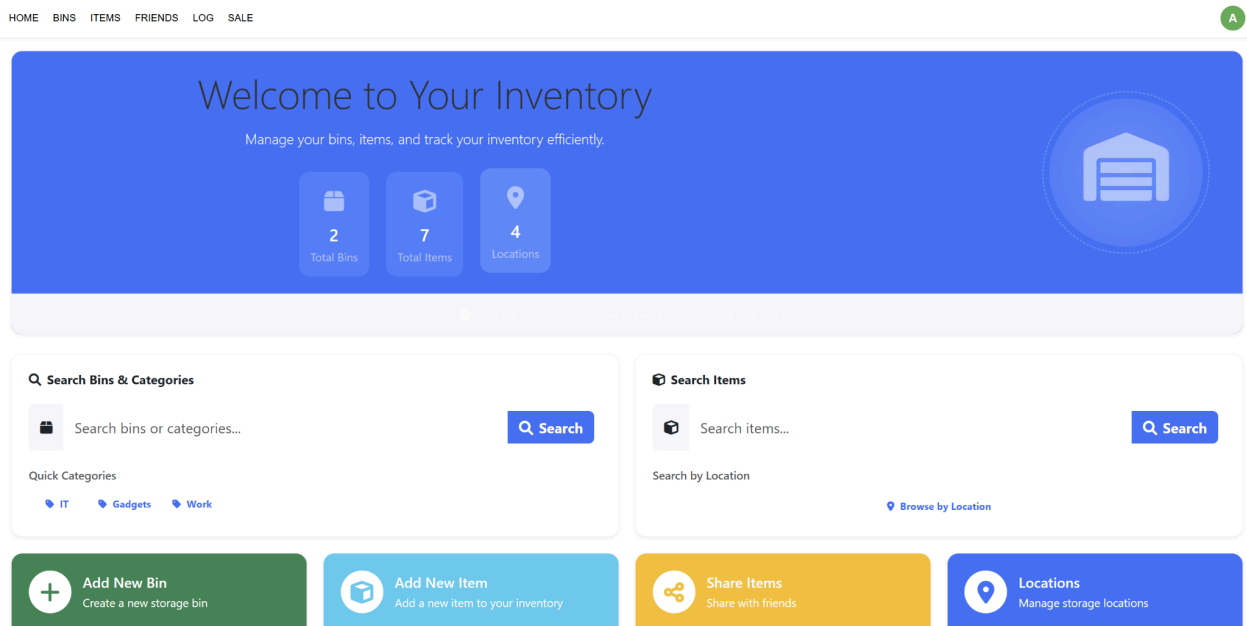
Two additional features that greatly enhance the user experience are the logout functionality and the navigation bar. To ensure users can securely exit their accounts, we included a user icon that, when clicked, provides a clear and safe option to log out. Additionally, to improve the overall usability of the app, we implemented a top navigation bar that gives users quick access to all major features. This allows for faster and more intuitive navigation between different pages and functionalities within the application.

Dashboard UI Mock-Up

Original design:



Current design:



2.6 Sharing and Friends:

One of the more challenging requests from our customer was the ability to share specific bins with other users. To implement this feature securely, we had to ensure that any user selected for sharing already existed in our database. The sharing process was designed in two steps to maintain both flexibility and security. First, when creating a bin, users can indicate whether it might be shared in the future. Selecting this option alone does not initiate sharing. Later, users can browse their list of friends and choose with whom they want to share the bin. Once shared, the selected friend will see the shared bin alongside their own bins. However, they will not have control over the sharing settings—they can only view, edit, or delete the shared bin.

Given the complexity of this feature, we initially marked it as optional after consulting with the customer and excluded it from our early UI designs. However, during our fourth sprint, we successfully implemented this functionality. Users can now add friends within the application and later choose which bins to share with them, providing a secure and collaborative storage management experience.

Feature: Mark Bin for Potential Sharing

As a user of the application,
I want to mark a bin as potentially shareable when I create it,
So that I can enable sharing later without editing the bin again.

Points: 2

Feature: Add Friends

As a user of the application,
I want to add other registered users as friends,
So that I can share specific bins with them later.

Points: 3

Feature: Share Bin with Friends

As a user of the application,
I want to choose a friend to share a bin with,
So that they can collaborate with me by viewing, editing, or deleting the shared bin.

Points: 5

Feature: View Shared Bins

As a user who has received shared bins,
I want to see shared bins alongside my own bins,
So that I can manage and interact with them seamlessly.

Points: 3

Feature: Restrict Sharing Control

As a user who has received a shared bin,
I want to be able to interact with the bin (view, edit, delete) but not change its sharing settings,
So that the original owner retains control over sharing permissions.

Points: 2

Sharing UI Mock-Up

My Friends

 Add New Friend

 Enter friend's email address

+ Add Friend

 Friends List

 rafaeldms27@icloud.com
Friend

 View Bins

 View Items

 Remove

Share "Networking"

Select Friends to Share With

Select All Deselect All



Search friends...



Rafael

rafaeldms27@icloud.com

✓ Currently shared

← Back to Bins

Update Sharing

2.7 Sales and logs:

In addition to the core features, our customer requested two advanced options to enhance the app's functionality: item activity logging and garage sale management. For logging, users can specify a custom time range by selecting a start and end date. The system then generates a printable report listing all items that were added, deleted, or edited during that period. This

feature helps keep track of inventory changes over time, especially for documentation or auditing purposes.

The second feature allows users to mark items for a garage sale. Each item includes an optional flag that can be enabled to indicate it's for sale. Additionally, users can filter and identify items that have remained in storage for more than a specified number of years, making it easier to declutter and prepare those items for sale. These two features provide both organizational insight and practical tools for managing and downsizing storage over time.

Feature: Activity Logging by Time Range

As a user of the application,

I want to specify a start and end date,

So that I can generate a printable report of items added, deleted, or edited during that time.

Points: 3

Feature: Generate Printable Activity Log

As a user of the application,

I want the activity report to be printable,

So that I can save or physically archive a record of inventory changes.

Points: 2

Feature: Mark Item for Garage Sale

As a user of the application,

I want to flag individual items as "For Sale,"

So that I can easily identify and manage items I plan to include in a garage sale.

Points: 2

Feature: Filter Long-Stored Items

As a user of the application,

I want to filter items that have been in storage for more than a specified number of years,

So that I can find and mark old items for potential sale or removal.

Points: 3

Log and Sale UI Mock-Up

LOG SALE

From: mm/dd/yyyy

To: mm/dd/yyyy

Search Logs

SALE

Created Before:

04/23/2025

Only Items For Sale

Search Items

4.

Sprint	Product Owner	Scrum Master	Developer-I	Developer-II	Developer-III	Developer-IV
I	Rafael Morales	Calvin Parker	Khush Patel	Saba Mostofi	Arun Akash Rangaraj	Sid Rayapur
II	Khush Patel	Rafael Morales	Saba Mostofi	Arun Akash Rangaraj	Sid Rayapur	NA
III	Saba Mostofi	Arun Akash Rangaraj	Khush Patel	Rafael Morales	Sid Rayapur	NA
IV	Arun Akash Rangaraj	Khush Patel	Saba Mostofi	Rafael Morales	Sid Rayapur	NA

5.

Sprint 1

Backlog Item	Description	Points
User Authentication	Implemented login, logout, and registration features.	7
CRUD for User	Developed CRUD (Create, Read, Update, Delete) operations for the User model.	6
CRUD for Bin	Developed CRUD operations for the Bin model.	6
CRUD for Item	Developed CRUD operations for the Item model.	6
Dashboard (Simple HTML)	Developed a basic Dashboard Page	6

Sprint 2

Backlog Item	Description	Points
Create Session Model	Tracks user login sessions, storing login timestamps and enabling date-range queries using the <code>between_dates</code> scope.	3
Create Location Model	Represents a user-defined place that groups multiple bins and items, supporting full CRUD functionality with automatic cleanup via cascading deletions.	3
Create Picture Model	Added support for attaching pictures to bins and items.	6
Add QR Code to Bin	Enabled QR code association for bins.	4.5
Add NFC Chip to Bin	Integrated NFC chip functionality with bins.	4.5
Prevent Item Deletion after Removal from Bin	Prevented unintended deletion of items from the database after removal from their respective bins.	4.5
Allow Items That Don't Belong to Any Bin	Allowed creation of stand-alone items that are not linked to any bin.	4.5
Password Recovery	Implemented password recovery functionality.	7
Improve Bin Form with Dropdowns	Added a dropdown menu to improve the search functionality for bins.	1

Sprint 3

Backlog Item	Description	Points
Update session Model	Applied required updates carried over from the last sprint.	3
Update Location Model	Applied required updates carried over from the last sprint.	3
Add Search Bar to Dashboard	Enables dynamic search functionality by Bins, Categories, Items, or Locations.	6
Enhance UI for Bins	Improved user experience with enhanced search and sorting by name.	3
Enhance UI for Items	Improved user experience with enhanced search and sorting by name.	3
Implement Location Routes	Displays all user locations; enables searching bins and items within them.	6
Add Pop-up Windows for Profile, Session, Log	Introduced pop-up windows for user profile, session, and log access.	6

Sprint 4

Backlog Item	Description	Points
Improve Dashboard Responsiveness for Mobile Screens	Fully implemented to ensure optimal display on mobile devices.	3
Enhance Responsiveness for Bins, Items, and Locations	Successfully optimized for better viewing and interaction on mobile screens.	12
Redesign Pop-up Windows for Mobile Screens	Redesigned and implemented for seamless mobile compatibility.	6
Mark Items for Sale	Tag items for sale that were created within a specified date range.	4
Add User Sharing Capabilities	Enabled shared access to bins, allowing users to view and edit collaboratively.	8
Fix Flash Message Handling	Fixed all JavaScript-related errors affecting flash	2

	message functionality.	
--	------------------------	--

6.

Sprint 1

Member	No.of Stories Attended	Points
Rafael Morales	2	6
Calvin Parker	0	0
Khush Patel	2	6
Saba Mostofi	1	6
Arun Akash	2	7
Sid Rayapur	1	6

Sprint 2

Member	No.of Stories Attended	Points
Rafael Morales	3	10
Calvin Parker	0	0
Khush Patel	3	9
Saba Mostofi	1	6
Arun Akash	2	7
Sid Rayapur	3	6

Sprint 3

Member	No.of Stories Attended	Points
Rafael Morales	3	8
Calvin Parker	0	0
Khush Patel	2	6

Saba Mostofi	2	6
Arun Akash	2	6
Sid Rayapur	1	4

Sprint 4

Member	No.of Stories Attended	Points
Rafael Morales	2	7
Calvin Parker	0	0
Khush Patel	1	6
Saba Mostofi	2	8
Arun Akash	2	8
Sid Rayapur	1	6

7.

Customer Meeting – 01/31/2025 (Hybrid Mode):

The customer outlined key requirements and expectations for the application, providing a clear direction for development. Based on this input, the team created the initial design diagram and user stories for Sprint 1, effectively laying the groundwork for future sprints.

Customer Meeting – 03/21/2025 (Zoom):

Presented the latest version of the application, featuring the Dashboard, My Bins, My Items, and My Locations pages. The customer emphasized the need for mobile-friendly layouts across all major pages and requested the ability to share bins, with both view and edit permissions, between users.

8.

The customer outlined their needs and expectations through user stories, which were translated into corresponding BDD scenarios. A paper prototype of the web page was created to align with these requirements, followed by the development of related TDD cases. The implemented web page was then thoroughly tested using the defined BDDs with Cucumber and TDDs with RSpec.

9.

GitHub served as the primary tool for configuration management throughout the project. Dedicated branches were created for each feature, and once reviewed and approved, they were merged into the main branch. Before starting work on any new feature, the latest version was always pulled from the main branch to ensure consistency and avoid conflicts, which were resolved as needed. A total of 23 branches were created—some tailored to individual developers for tracking progress and experimenting with tools and integrations such as QR code functionality, password recovery, log history, and user sharing. Over the course of the project, four stable versions were released—one at the end of each sprint.

10. Production Deployment Issues

The process of deploying the app on Heroku went smoothly. One of the challenges we faced was implementing SMTP email support. Every user needs to receive an email in order to reset their password, so setting this up was crucial. We spent several hours figuring out the correct configuration to use an email account in production, especially due to Gmail's specific requirements. This involved understanding environment variables and the differences between test and production environments. Aside from this, we didn't encounter any issues pushing our code to Heroku.

11 . Tools Gems Used

- **Devise:** This is a flexible authentication solution for Rails. Initially, we started our app using the built-in authentication features of Rails. However, we wanted a more robust and secure authentication system. Devise allowed us to implement features such as password reset and account recovery with ease.
- **qrbarcode:** This gem is used for generating and rendering QR codes. It was essential for embedding unique URLs into QR codes for each bin, enabling users to quickly access bin information by scanning the code.
- **Testing Stack:** We used RSpec, Cucumber, Capybara, and SimpleCov for testing. This stack allowed us to write both unit and system tests effectively, simulate user interactions, and measure test coverage to ensure application reliability.
- **Kaminari:** This gem helped us implement pagination on the front end, improving performance and user experience when dealing with large datasets.
- **image_processing:** This gem was used to process images associated with bins and items. It works in conjunction with Active Storage, allowing us to resize, crop, and optimize images for consistent display across the platform and to reduce file size for faster loading.

12. All the cucumber scenarios and RSPEC code are included in the main repository

13. Guideline for app deployment:

Everything needed to deploy the Inventory Control App is included in this repository. There are no external libraries or dependencies beyond what is already specified in the repo.

In order to deploy the code, you will need a Heroku account. Deployment requires subscribing to the lowest-cost Heroku tier for dynos and a paid Heroku Postgres database plan, as the free tier no longer supports production apps with persistent storage.

If this project is handed off to a new team in the future, they will need to configure their own production environment—particularly the SMTP email configuration. We used a dedicated Gmail account to handle password reset functionality. Due to security best practices, the credentials for this account are not stored in the repository and must be set up separately using environment variables.

The SMTP setup is straightforward. Future teams or the customer can decide whether to continue using a Gmail-based setup or switch to another email service.

The application is fully functional and deployable as-is, but it is still considered a prototype. The final decision on whether to adopt the app, scale it, or integrate additional services will be made by the customer based on their needs.

14.

- Deployed app: <https://robert-inventory-f91e26f91bb2.herokuapp.com/>
- Github Project: <https://github.com/orgs/tamu-edu-students/projects/101>
- Github Repo: <https://github.com/tamu-edu-students/CSCE-606-Inventory-System>

15.